Fraud Detection Under Extreme Class Imbalance: A Comparison of Supervised and Unsupervised Methods

Sawyer Mckenney
Boulder, Colorado, USA

Samuel Meaux
Boulder, Colorado, USA

David Savić
Boulder, Colorado, USA

Abdirahman Abdi
Boulder, Colorado, USA

Abstract

The goal of this project is to evaluate and compare supervised and unsupervised fraud detection methods for identifying fraudulent financial transactions under extreme class imbalance. Specifically, we aim to answer the question "How do supervised fraud detection Models compare to unsupervised anomaly detection methods in identifying rare fraudulent financial transactions and under what conditions does each approach perform best?" Fraud detection appears to be a big problem in the finance world, where undetected fraud can lead to significant monetary losses, while false positives can negatively impact customer experience. Financial institutions, payment processors, and online merchants can all benefit from effective fraud detection systems. By comparing supervised and unsupervised approaches, this project provides insight into trade-offs between Model Accuracy, interpretability, and reliance on labeled data, which is often scarce or costly to obtain.

We built and evaluated many supervised and unsupervised methods on the Feedzai BAF dataset under an extreme amount of class imbalance. Ensemble methods, primarily Stacking achieved the strongest performance, whereas unsupervised learning techniques were more useful for discovering patterns as opposed to actually detecting fraud.

1 Introduction and Related works

It is getting more and more important to spot financial fraud, especially with so many digital transactions happening every day. Reports suggest that globally, we are losing over twelve billion dollars each year just from payment fraud. This really highlights why we need much better ways to find fraud, going beyond the old systems that just use fixed rules and expert guesses. Those older methods are not as good anymore because fraudsters are always finding new tricks with technology. That is why more and more people are turning to machine learning to pick out fraudulent patterns in financial data.

One of the biggest headaches in fraud detection is that actual fraud is really rare. In most real-world situations, less than one percent of all transactions are fraudulent. This means that if you just look at how "accurate" a system is, it can be pretty misleading. For instance, a Model could say every transaction is fine and still seem over 99% accurate, even though it completely misses all the fraud. Earlier research has shown that it is much more helpful to look at things like Recall, Precision, and F1-Score(Dal Pozzolo et al., 2015). These measures do a better job of showing the balance between catching fraud and not flagging too many legitimate transactions by mistake.

When we are trying to solve these issues, people have looked at two main types of machine learning: supervised and unsupervised. Supervised Models, which include techniques like logistic regression or decision trees, are really effective when you have plenty of data already marked as fraudulent or legitimate. This means you know exactly which transactions are fraud. The tricky part is, we often do not have a lot of perfectly labeled fraud data. Plus, Scammers are always finding new ways to work around detection, so what was fraud yesterday might look different today. That is where unsupervised methods come in. Tools like clustering or anomaly detection can spot unusual patterns and behavior without needing to be pre-trained on what fraud looks like. They just look for things that stick out. Both of these approaches have their good sides. Knowing when to use one over the other is actually pretty important for building effective systems to catch fraud.

This particular project is all about comparing supervised fraud detection Models with unsupervised anomaly detection techniques, especially when there is very little fraud data mixed in. We want to see how well each approach can find those rare fraudulent transactions and figure out which method works best under different circumstances.

When it comes to catching fraud, especially with credit cards or in banking, people have often turned to a method called supervised learning. For instance, a study by Bhattacharyya and his team back in 2011 showed that certain Models, like logistic regression, decision trees, or random forests, could actually do a good job at spotting fraud. This worked when they were taught using data where it was already clear what was fraudulent and what was not. More recently, folks have found that if you put several of these Models together, using tricks like boosting or Stacking, you can get even more accurate and reliable predictions. The tricky part with all these methods, though, is that they really depend on having a ton of good, pre-labeled data, which, in the messy real world of fraud, is usually pretty tough to come by.

Since finding enough labeled data can be a real challenge, some researchers have explored unsupervised approaches. These methods, like clustering or anomaly detection, aim to find transactions that simply do not fit normal patterns. Chalapathy and Chawla, in 2019, highlighted how anomaly detection, even with more advanced deep learning, is increasingly used to find those rare, unusual events in datasets where fraud is very uncommon. While these methods are good for uncovering hidden patterns, they often are not as precise as supervised Models when you are trying to make a direct "this is fraud" or "this is not" decision.

Another angle that is been explored for fraud detection is called association rule mining. Methods like Apriori, developed by Agrawal

and Srikant a long time ago in 1994, are used to find frequent patterns and connections between different things. These techniques can help make the rules more understandable to us. However, they can also struggle in datasets where fraudulent events are just too rare to create strong patterns.

From all the research, it seems pretty clear there is not one perfect way to catch fraud. Instead, putting different methods together, like mixing both supervised and unsupervised approaches, usually works much better. This helps systems not only predict fraud well but also gives us a better grasp of the tricky ways fraudulent activities tend to show up.

This paper goes on to compare supervised and unsupervised methods on the BAF dataset that contained an extreme amount of class imbalance. We evaluated how classification Models, clustering methods, association rule mining, and Ensemble methods perform in terms of Recall, Precision, F1, and usefulness in real time operations. We show how Ensemble supervised methods provided the strongest predictive performance, while unsupervised methods are most useful for understanding structural behavior.

1.1 Project Description

The goal of this project is to evaluate and compare supervised and unsupervised fraud detection methods for identifying fraudulent financial transactions under extreme class imbalance. Specifically, we aim to answer the question "How do supervised fraud detection Models compare to unsupervised anomaly detection methods in identifying rare fraudulent financial transactions and under what conditions does each approach perform best?"

At a high level, we will perform predictive Modeling and anomaly detection analysis using transaction-level data. Supervised learning methods such as logistic regression, random forests, and gradient boosting will be trained using labeled fraud data. In contrast, unsupervised methods will be applied to identify anomalous transactions without using labels. This will also help us identify patterns that we may not have previously noticed in the labeled data. Model performance will be systematically compared to understand the strengths and limitations of each approach in real-world fraud detection scenarios.

1.2 Data Sources

Credit Card Fraud Detection

https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud (284k transactions with 30 features and binary fraud label) [10]

The Fraud Dataset Benchmark (FDB):

https://github.com/amazon-science/fraud-dataset-benchmark (This is a compellation of data sets that we found on amazon science) [2]

Based on our limited research, common variables used to detect fraud include transaction amount, transaction type, and currency.

1.3 Potential Bias and Limitations

The most glaring hurdle we ran into with the Feedzai BAF dataset was the massive class imbalance. Actual fraud makes up a tiny fraction of the data—roughly 1.1%. It became obvious pretty quickly that standard Accuracy was going to be a useless metric for us. A completely lazy Model could just label every single application as legitimate. This would ultimately result in a great 98.9% Accuracy, but would spectacularly fail entirely at its actual job. Because of this, we had to rethink the approach and decided to eliminate Accuracy as a qualifying metric and primarily rely on Recall, Precision, and F1-Scores to see if our Models were actually catching bad actors.

Another factor we had to keep in mind is that the data is entirely synthetic. It was originally generated for a 2022 NeurIPS paper to mimic real banking fraud without violating consumer privacy laws. While the privacy aspect is necessary, synthetic data naturally struggles to replicate the truly chaotic, unpredictable tactics that real fraudsters invent on the fly. Realistically, the Ensemble pipelines we built here serve as a proof-of-concept. If a financial institution actually wanted to deploy this system, the Models would absolutely need to be retrained and validated on messy, live production data first.

We also had to make an educated decision regarding missing data. Huge chunks of the history-based columns, like prev_address_months_count and bank_months_count, were packed with "-1" placeholders. As a data scientist, the standard move is to either drop those rows entirely or swap the placeholder for a median value. But doing that felt like throwing away huge clues. We operated on the theory that if an applicant completely lacks a banking or residential history, that isn't just bad data—it is a major red flag for a synthetic identity. Instead of erasing that context with imputation, we built indicator flags to track exactly what was missing. This let our Models use the absence of data as a core predictive signal rather than treating it as noise.

1.4 Problem Motivation

Financial fraud detection is a serious problem in our modern economies due to the volume of online transactions and the fact that they will only continue to increase. Electronic payment systems also contribute to this. As transaction volumes increase, so does the opportunity for fraud to take place, so it is imperative to find methods to prevent and detect these behaviors. A key challenge in fraud detection is extreme class imbalance, where fraudulent transactions represent only a very small fraction of all activity. This imbalance complicates traditional machine learning approaches and makes naïve performance metrics misleading. Many fraud detection systems must identify a handful of malicious cases among hundreds of thousands of legitimate transactions. Missing fraudulent activity can result in substantial financial losses, while overly aggressive detection can disrupt normal customers. Beyond the immediate monetary loss, the persistence of undetected fraud is one of the key factors in the erosion of trust in any digital banking infrastructure. As those perpetuating this fraud are becoming more sophisticated with the increases in technology, "rule-based systems" are starting to become increasingly more out of touch with reality. This now requires programmers and security professionals to move toward dynamic, data-driven mining techniques that can adapt to evolving patterns. As a result, designing effective fraud detection systems requires careful trade-offs. This project focuses on comparing supervised fraud detection Models with unsupervised anomaly detection techniques under severe class imbalance. We will look at how these approaches fluctuate in their ability to detect fraud transactions. Understanding these differences is required for building real-world fraud detection that is both practical and scalable.

1.5 Data Collection

During the data collection step, we opted not to use a dynamic API to collect our data for many reasons. For starters, there are latency challenges, security challenges, as well as overall availability issues that present themselves when utilizing an API. Getting a business, or anyone in fact, to agree to give us real-time access to live streamed transactions is a major security risk and would cost any company money. During this process, we also explored the idea of using real-time crypto transactions as our data source but decided not to go this route. This came up because this would have given us a real time transactions feed but did not provide a fraud label. Without labels, we would not have been able to evaluate supervised Models or fairly compare these Models with unsupervised Models.

While we were not able to secure a live data stream, we were still able to pull our chosen dataset, the bank account creation fraud records, directly through a public API. The data set we ended up using was the Bank Account Fraud (BAF) dataset by Feedzai. We discovered that Feedzai is an actual fraud detection tool that works with various institutions. The implication here is that Feedzai and the dataset we discovered is not just a random CSV file. This data set was actually introduced in a NeurIPS 2022 paper [9] and was designed to be built by using privacy preserving methods that were applied to real bank account opening fraud data. Since this data is synthetic, it does not mean it is unusable because it was Modeled after real fraud behavior patterns and designed to be used specifically for training machine learning Models in a financial setting, just as we are doing now.

This data set fits our research question perfectly, because this entire project is about comparing supervised fraud detection Models to unsupervised anomaly detection under extreme class imbalance. The BAF dataset has transaction-level features and a clear fraud label (fraud_bool), which lets us train supervised Models and test anomaly detection methods side by side. The data set as expected is extremely imbalanced with only about 1% of the data being fraud, and this mimics a real world setting. This imbalance is quite literally the core challenge we are studying in this project, so the data set supports the goals of our project without us having to manipulate the data artificially.

1.6 Data Understanding

Each row in the data set represents a single application for a bank account. Each account application was submitted by a user who was attempting to open a financial account or line of credit. This data set has numerical features, categorical features and binary features. The numerical features in this data set include income, customer_age, credit_risk_score, session_length_in_minutes and proposed_credit_limit. All of these are great numerical variables that can be used in analysis and in various machine learning Models. Some of the categorical features include payment_type, employment_status, housing_status and device_os. These variables will require encoding methods such as one-hot encoding before they can be used in machine learning algorithms. Finally, we have binary features that can only be 0 or 1. Some examples of those data include fraud_bool, email_is_free, phone_home_valid, foreign_request and has_other_cards. Since binary variables are already numeric, we will not need to require additional encoding on these values.

1.7 Data Cleaning and Preprocessing

The goal of this phase was to get the dataset (Feedzai) to a state where it can be used prior to Modeling. We came across a mix of numerical and categorical fields in the raw file, as well as columns which have a placeholder for missing values of −1. We converted those to proper missing values so that they could be handled correctly.

Numeric columns were filled using the median since several features are skewed and the median is less sensitive to extreme values. Categorical columns were filled with the most common category in each column. After that, numerical features were standardized so they are all on a similar scale and categorical features were one-hot encoded so they can be used later in machine learning Models.

To save the changes, we saved the first ten rows of the dataset before and after cleaning. These snapshots show how missing values were handled and how categorical variables expanded into multiple columns after encoding, We also generated several visual checks (fraud distribution plot, correlation heatmap, payment type frequencies, velocity histograms, and QQ plots for normality). These helped confirm what we saw in the summary statistics – skewed distributions, redundant velocity features, and no single feature that strongly predicts fraud on its own.

We have exported the final result as cleaned_feedzai.csv and will be using it in the Modeling stage in Milestone 3.

1.8 Data Quality, Bias and Limitations

The data is complete in general, however, fields like prev_address_months_count and bank_months_count contain large gaps. These missing values are not random and may actually describe a customer that is new or has little history, which itself could be relevant to fraud detection. The dataset is also synthetic, which protects privacy, but may not capture every detail of real-world fraudulent behavior. Another important limitation is the class imbalance: fraud accounts for only about 1.1% of the applications, making it easy for a Model to look accurate while still missing most fraudulent cases.

1.9 Modeling Overview and Approach

The challenge of our selected project was creating Models that were able to detect fraud in a dataset where only 1.1% of all transactions were fraudulent. In a dataset of nearly 1 million transactions, a Model that ignored all fraud and labeled all transactions as non-fraudulent would perform with 98.9% Accuracy. The shared goal for this project was to prioritize "Recall" (catching fraud) over "Accuracy." For our project approach, our team implemented four distinct Model combinations (or stacks), leveraging various methods and combinations to detect fraud bank account applications within the Bank Account Fraud (BAF) dataset by Feedzai. Each member of the team implemented their own algorithm and technique. By doing this, we were able to compare supervised classification, along with unsupervised clustering and Ensemble methods. Our approach includes Decision Trees, Support Vector Machines (SVM), Logistic Regression, K-Nearest Neighbor (KNN), K-Means Clustering, Apriori Association Rule Mining, and combination methods such as Stacking and Soft Voting. This has allowed our team to evaluate which approaches are more effective at handling class imbalances.

1.10 Model Selection

Prior to starting this project, we examined the root challenge of the problem with detecting fraud in a highly unbalanced dataset. Instead of simply choosing and sticking to a single approach, we decided to choose Models that have different strengths in an effort to compare approaches under such conditions and determine the best Model for our data. We chose to use Decision Trees because they give clear, rule-based splits that help us understand what features might contribute to fraud predictions. This method also does not require as much preprocessing and is simpler to use with mixed feature types.

Logistic Regression was chosen as a highly scalable, probability-based approach. Instead of creating rigid rules, it calculates the statistical likelihood of an account being fraudulent, allowing it to run in extremely fast linear time (O(n)). This allowed us to test if a flexible, statistical threshold could effectively identify rare fraud cases without bogging down our system on a massive dataset.

Support Vector Machines (SVM) were chosen to separate data into groups methodically where there are a lot of features. In this case, our raw data set had over 30 features (both quantitative and categorical). The SVM method helps to catch fraudulent transactions in certain cases where a straight-line method might miss them.

K-Nearest Neighbors (KNN) was used as a distance-based approach. This Model does not really "learn" one big Model ahead of time. Instead, it compares the new tuple to the stored data and decides based on similar cases which group it belongs to. This allowed us to test if local patterns were capable of identifying fraud.

Stacking and Ensemble Methods were used to combine the different strengths of individual Models into a single Model. Soft Voting along with Stacking was used to intelligently determine the weight of Model outputs. Both of these Models aimed to improve the overall robustness of the Model.

K-Means Clustering was an unsupervised learning approach we decided to test out, not expecting it to provide very much information because of the nature of the data we are dealing with. It was used to see if fraud appears in identifiable segments.

Apriori Association Rule Mining was used because it helps identify easy to understand patterns in data instead of predicting. For example, we were curious to see if it was able to show if parameters like high credit risk and foreign requests appear together or not. This method helps provide very valuable insights into fraud analysis.

1.11 Performance Evaluation Metrics

To evaluate the performance of each of our Models, we first needed to understand that a Model with a 98.9% Accuracy is very misleading. So, to evaluate our Models correctly we relied on the following metrics:

Recall: the proportion of actual fraud cases correctly identified. This is arguably the most important metric because a missed instance of fraud correlates directly to a financial loss. A high Recall means that fewer fraud applications slip through the cracks.

Precision: the proportion of predicted fraud cases that were actually fraud. If a Model had low Precision, it means that there was an extremely high false alarm rate.

F-1 score: the consideration of both Precision and Recall. This score provides a single balanced measure. Very useful for comparing Models that trade off these metrics.

Accuracy: the overall proportion of correct predictions. While we include this for completeness, Accuracy is the least informative metric for this problem due to the extremely large amount of class imbalance our data set faces.

Confusion Matrices: a breakdown of true positives, true negatives, false negatives, and false positives. It essentially shows where each Model fails, so we can understand the tradeoffs of each approach.

2 Model 1 (Samuel Meaux): Class Imbalance Handling using Decision Tree, Logistic Regression, Support Vector Machine, Ensemble and Stacking

For this analysis, an initial exploratory phase took nine distinct Modeling architectures and evaluated them in order to determine the best method for identifying fraudulent account creations. Ultimately, the scope was narrowed down to five primary Models: Decision Tree, Support Vector Machines (SVM), Logistic Regression, a Soft Voting Ensemble, and a Stacking Ensemble. The main objective of this subset of Models was to test how these algorithms performed when they were trained on an artificially balanced dataset and then subsequently evaluated on a largely imbalanced real-world dataset.

The pipeline that was implemented for preprocessing was designed to identify behavioral signals from incomplete data. The dataset had placeholder values of '-1' and those values were replaced with standard 'NaN' values. Instead of filling in all missing data, explicit "missingness" indicator flags were created for the features "prev_address_months_count" and "bank_months_count." This approach was operating on a hypothesis that missing financial or residential history may not just be random noise or faulty data. It may very well be a behavioral feature that indicates a false identity or a fraudster that attempts to hide their misdoings. Following the imputation of legitimate missing values, LassoCV was utilized for feature selection. The Lasso function applies a regularization of "L1", which is responsible for penalizing overly complex Models. It does this by shrinking the coefficients of less statistically significant predictors to zero. This only reduced the Model from 49 to 47 highly significant features, however, ensuring that the following "heavy" algorithms ran as efficiently as possible.

Management of the extremely imbalanced (1.1%) fraud class required calculated resampling methods. Initially, Random Oversampling was attempted to bring the minority class volume up to that of the majority class. However, trying to train the SVM on nearly one million rows of data proved computationally impractical for the system specifications the Model was being run on. SVMs inherently are designed with a complexity that ranges from $O(n^2)$ to $O(n^3)$, which meant that the oversampled data was causing the Model to stall out and fail to converge on a solution within a timeframe that was remotely reasonable or even applicable in a real-world environment. Consequently, Random Undersampling was implemented instead, in order to reduce the majority class volume to match that of the minority. This resulted in a decently manageable and balanced training set of 11,029 samples per class.

This was incredibly important in preventing the Models from just ignoring the rare cases of fraud.

2.1 Computational Complexity Considerations:

A massive roadblock we ran into during this analysis was the sheer computational time and space required by our chosen algorithms. For example, SVMs have a training time complexity that scales terribly—anywhere from $O(n^2)$ to $O(n^3)$ (where n is the number of rows). When we tried to oversample the dataset up to nearly a million rows, calculating the SVM kernel matrix completely maxed out our system's memory and created a huge bottleneck. Decision Trees were a bit easier to utilize in this regard. With a complexity of $O(n \log(n))$, they fell more in the middle, but still took a lot of computational power on the massive dataset. On the flip side, Logistic Regression runs in linear time ($O(n)$). This was more mathematically efficient and is exactly why it ended up being so quick to train and and so easy to scale to the full, imbalanced dataset. Ultimately, because our Stacking Ensemble could only run as fast as its slowest base Model (the SVM), we had to pivot. Using Random Undersampling to shrink the dataset down to 22,058 rows was pretty much the only mathematically viable way to train our pipeline without needing access to a supercomputer.

2.2 Real-World Execution Times:

To put these computational challenges into perspective, we tracked the actual execution times for our entire pipeline. Even after using Random Undersampling to shrink the training set down to a highly manageable 22,058 total samples, the processing was still intensive. The initial data cleaning and preprocessing phase alone took roughly 177 seconds. Training the entire suite of algorithms, including the bottleneck SVM and the final Stacking Ensemble, took 204 seconds. The most computationally intensive portion of this section, however, was evaluating these trained Models on the full, 1,000,000 row testing set. That portion by itself took over 1,334 seconds (approximately 22 minutes). In total, the full notebook took nearly 29 minutes (1,715 seconds) to execute. These hard numbers are a clear depiction of exactly why attempting to oversample the data for $O(n^2)$ and $O(n^3)$ algorithms would have (and did, initially) completely crashed our system.

2.3 Issues, Improvements, and Conclusion:

Through the combination of feature optimization and Ensemble methods, there was a significant improvement in fraud identification on the balanced test data. The Stacking Model, which was able to intelligently weigh the outputs of the underlying SVM, Decision Tree, and Logistic Regression Models, was effective at capturing nearly 80% of fraudulent activity (79% Recall).

However, a fairly critical difference in performance was observed when those same Models were evaluated against the full, imbalanced testing set. While Stacking and SVM still caught a large portion of the fraud (maintaining high Recall), their Precision tanked to about 2%, dragging their F1-Scores down to just 4-5%. The most surprising takeaway was how well the standalone Logistic Regression Model handled the extreme class imbalance. Since it works off probabilities instead of hard cutoffs, it managed to keep its Recall at 80% while also hitting the highest Accuracy (over 80%) and the best overall F1-Score (8%) on the full test set.

Despite this success, a large portion of the non-fraudulent bank account initiations were still flagged as fraudulent across all Models. Future iterations of these Models must take a look at implementing more advanced threshold tuning and maybe even take a deeper dive into non-linear deep learning techniques in order to reduce the high volume of false flags while not sacrificing the incredible Recall rates achieved in these Models.

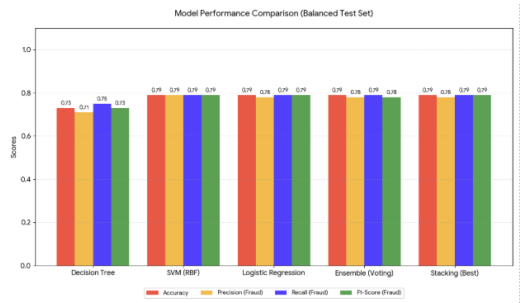

Figure 1: Model Comparison of Performance (Balanced Test Set)

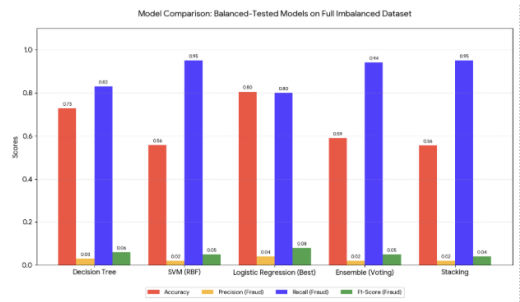

Figure 2: Model Comparison (Balanced-Tested Models on Full Imbalanced Dataset)

3 Model 2 (Kayo Abdi): Fraud Detection using Decision Tree, K-Means Clustering, and Apriori

For this set of Models, the dataset was preprocessed by replacing values -1 with NaN to represent data that was missing. For numerical features, median imputation was applied. For key attributes such as bank account durations and previous addresses, however, missing values were treated as meaningful indications rather than just noise. One-hot encoding was used on categorical variables, which gave a total of 47 features and the final dataset contained just 1,000,000 observations.

Three different Modeling methods were used including Decision Tree classification, K-means clustering, and Apriori association rule mining. Each Model was selected to capture different aspects of

fraud detection. This included behavioral grouping, predictions, and pattern discovery.

The decision tree Model was chosen due to its easy interpretability and suitability for classification. This Model assumes that the data can be partitioned into different decision regions based on feature thresholds. To reduce overfitting, the maximum tree depth was limited to 5. Also, due to the severe class imbalance of about 1.1% fraud, class weighting was applied to highlight the minority class (aka fraud). The Model achieved a Recall of around 74% for fraud cases, thus representing effective detection of fraudulent activity. But Precision was low by around 3%, reflecting a high number of false positives. The confusion matrix shows that most non-fraud cases are correctly classified, but a large number are incorrectly predicted as fraud, while only a smaller portion of fraud cases are missed. After some research, it was discovered that this trade-off is expected in fraud detection scenarios where minimizing missed fraud cases is prioritized. This suggests that the Model is more focused on capturing as many fraud cases as possible, even if it means incorrectly flagging legitimate users, which aligns with the goal of reducing financial risk. While the decision tree focused on directly predicting whether fraud happened or not, the unsupervised Models helped us step back and better understand how the data is organized. The K-means clustering Model grouped transactions into different behavioral segments, and when we compared those clusters to the fraud labels, we noticed that the fraud was not spread evenly across the data. Instead, it showed up more in certain clusters. That implied that fraud is not random. It is tied to specific patterns of behavior. Because of that insight, identifying and keeping an eye on these higher-risk groups could be just as useful as predicting individual transactions. Practically, it gives a way to catch suspicious activity earlier on a broader level.

The Apriori results took this a step further by looking at how features showed up together. Even though fraud is rare, which made it harder to find strong rules directly tied to it, the Model still picked up on combinations of features that appear frequently. What stood out here is that fraud does not seem to come from one single factor, but rather from multiple things happening at once. This lined up with what we saw in the decision tree, where no single feature managed the result. However, several features worked together to differentiate fraud from non-fraud cases. Combining all of this together, it became clear that no single Model can tell us the whole story. Therefore, using a mix of prediction, clustering, and pattern discovery gave a much better picture of both who is likely to commit fraud and the situations where fraud is more likely to happen.

4 Model 3 (David Savić): Fraud detection using Decision Tree, Naïve Bayes, Apriori, and Logistic Regression

Four Models were implemented on the Feedzai Bank Account Fraud dataset (999,999 rows, 1.10% fraud rate). Similar to the Models above, the same preprocessing pipeline was used throughout: placeholder "-1" values replaced with "NaN", median imputation on numeric columns, most-frequent imputation and OneHotEncoding on categorical columns, and StandardScaler on numeric features – yielding 51 features after encoding. An 80/20 stratified split was then applied across all Models.

The Decision Tree was the first choice because of the simplicity of output interpretation and explanation. Every prediction can be traced back to a specific split on a specific feature, which is extremely important when trying to justify a fraud flag. Decision Trees also handle the mix of numeric and encoded categorical features without much extra setup. Additionally, "class_weight" lets you penalize the Model directly for missing fraud cases.

Naïve Bayes was included mainly because it gives a fraud probability score rather than just a binary label. That is useful in practice – a fraud team can lower the threshold to catch more fraud at the cost of more false alarms, or raise the threshold to reduce noise. Retraining is not necessary; just shift the cutoff.

Apriori is not a predictive Model – it does not output a fraud label or score. The point of including it is to find which feature combinations tend to appear together in fraudulent applications and express them as readable rules. That is actually more useful for analysts than a black-box classifier in some cases.

Logistic Regression was included as the linear baseline and also because it is used as the meta-learner in our Stacking Ensemble. It outputs well-calibrated probabilities across the full range, which is what makes it a good choice for combining base Model outputs. L1 regularization can also zero out irrelevant features automatically across the 51 columns.

Both Ensemble methods combined all eight Models from the team: the three above plus the Linear SVC and KNN from Model 4, and the Random Forest and Gradient Boosting from Model 1. The training set was resampled to balance the classes first (RandomUnderSampler, 8,823 samples per class). KNN and RBF-SVM were trained on a 20% subsample of the balanced set because full training was too slow.

4.1 Model 3 Conclusion

The highest F1-Score of 0.0897 along with highest Receiver Operating Characteristic Area Under the Curve (ROC-AUC) of 0.8955, and highest Recall of 81.78% are a result of the Stacking Model. Gradient Boosting and Logistic Regression outputs were weighed more heavily from the meta learner, as the two had strongest individual performance. Not too far behind is Soft Voting with Recall of 81.37% and ROC-AUC of 0.8846. Both Ensemble methods outperformed every individual Model on every metric, and that is confirmation that merging the approaches is far better than relying on any of the ones as single method.

Besides the high performance of the Models, we also had to think about how much computational power each one would require. The Decision Tree was quite efficient, both for training and for making predictions. That made it a sensible basic Model to use. KMeans, on the other hand, needed more processing. That is because it constantly adjusted the cluster assignments across the dataset until it found the right groups. But Apriori really ate up the most computing resources. It had to go through a huge number of potential feature combinations just to find patterns that happened often, and what this really shows is that there is an important give-and-take: you can often get much deeper insights from more complex methods, but you will always have to deal with the higher computational cost that comes with them.

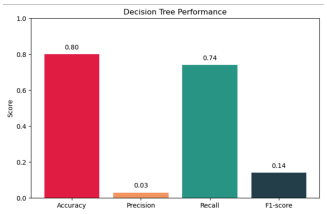

Figure 3: Decision Tree performance showing high Recall but very low Precision, illustrating the tradeoff between detecting fraud and generating false positives in an imbalanced dataset)

5 Model 4 (Sawyer McKenney): Fraud Detection using SVM, KNN, and Stacking

Linear SVM was chosen because after one-hot encoding, 51 dimensions were present. SVMs are designed to find decisions boundaries in high dimensional amounts of data. Since this project dealt with a large amount of class imbalance, and because it penalizes misclassifying the minority more heavily, SVM was assumed to be great for this purpose. As a result, SVM was able to achieve a high Recall of 0.78.

KNN was chosen because it does not make assumptions based on the distribution of the data. Instead, it focused on localized irregularities that an SVM might have missed. As a result, it was assumed that if these methods were stacked, it would be possible to leverage the strengths of both SVM and KNN. When it was observed that Stacking provided the highest F1-Score, these assumptions were proven to be correct.

For the SVM Model, "class_weight" was adjusted to be "balanced" so that the Model did not predict "non-fraud" for everything. The max iterations was also adjusted to be 5000 instead of 1000 to give the optimizer enough iterations to converge on the large dataset. For KNN, the only hyperparameter change that was made consisted of using all available CPU cores for faster computation by running in parallel.

During the preprocessing of this Model, the "-1" value was replaced with "NaN" for the features with large amounts of missing data. "-1" and "NaN" were the values that represented missing values. Secondly, median imputation and normalization were applied in other features. One-hot encoding was also used on categorical features, and we resulted in 51 features after encoding. The techniques applied in this Model include Normalization and Bagging. Normalization was required for a distance-based Model approach to apply to SVM and KNN.

Bootstrapping was used to try and reduce variances through aggregation, and it was applied to both SVM and KNN. For training, a 20/80 split was implemented, training by randomly selecting 80% of the data and then testing on the 20%. As a result, it was found that SVM had achieved the highest Recall with 77% but very low Precision of about 4%. This means that the Model was catching a large amount of fraud but also generating a lot of false alarms. On the other hand, KNN struggled with class imbalance and presented a 41% Recall and 4% Precision. After Stacking, Precision was able to be improved by 7% over the individual Models, and was able to balance Recall to 60%. Stacking also provided the best F1-Score for this Model at 0.13.

While developing this Model, we were able to better understand how linear SVM works to separate fraud and non-fraud by learning a decision boundary. This means that instead of looking at small groups of similar transactions SVM makes the assumption that there is a structure within the data set that allows us to differentiate fraud events. Since this data set contains 51 features that means that this Model is learning to separate fraud from non-fraud across a lot of variables at once.

This Model was able to achieve a high Recall compared to our other Models. This leads us to the idea that there is a consistent behavior across features in how fraud applications appear compared to non-fraud applications. This is subtle but important because it means that fraud is not completely random in the data set. This Model was able to identify some of these key characteristics and learn to identify them.

Since there was such a lower Precision, this shows the limitations of only looking at the global pattern. The Model was able to capture the general fraud behavior but at a cost of inappropriately flagging a lot of legitimate transactions. This shows us that not all fraud follows the same global pattern and it is also important to consider some localized distinctions. This is where KNN proved valuable. While the SVM successfully captured broad, global patterns in the data, it struggled to identify subtle, local irregularities. KNN naturally filled this gap, excelling exactly where the SVM fell short.

This really highlighted the gap between a Model's raw detection power and its actual usability in the real world. If a system flags too many legitimate transactions, it creates a massive backlog of manual reviews for human investigators. On the flip side, the financial cost of letting actual fraud slip through is incredibly high, forcing a very difficult balancing act. Besides just Accuracy, run-time performance is another metric that is worth considering. The SVM Model trained in 3.87 seconds on 160,000 samples and then was able to provide predictions on the test set within 0.02 seconds. This makes this Model extremely well-suited for real-time scoring pipelines. KNN on the other hand, is a lazy learner as it requires a bit more time to predict because it needs to search the training set at the time of the prediction. As the data set grows, this will only get worse. As a result, we were able to determine that SVM is effective as an initial filter, but not the most practical standalone solution. That is why we started Stacking various methods to understand how to leverage the strengths of different learning techniques.

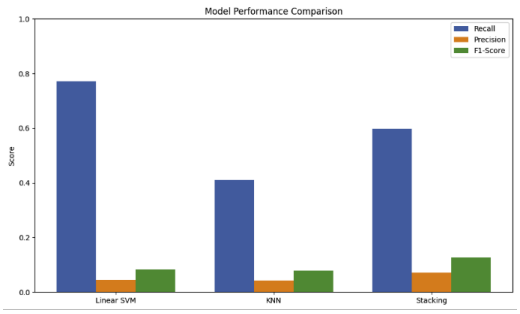

Figure 4: Model Comparison (Imbalanced Test Set)

5.1 Model 4 Conclusion

KNN was the weakest predictive Model in Model 4's implementation, failing significantly with class imbalance and producing poor Recall and uncertain performance overall. The standalone Decision Tree was also a consistent underperformer compared to other Models, especially in terms of maintaining a balance between Precision and Recall. In the end, the Ensemble methods used elsewhere in this study were proven necessary for the improvement of fraud detection within our scope.

6 Conclusion and Future Work

6.1 Future Work

Our current Ensemble method has successfully identified a majority of the fraud in the dataset. However, there are several approaches that are still untested that could be used in any future research projects in an attempt to improvise on the applicability of these Models in a real-world setting. First, the data used in this study was a synthetic dataset representing historical bank account fraud. We know that fraudulent activity is becoming more technically advanced and changing on a rapid scale as new technologies are developed. Because of this fact, future projects of this nature should take these Models and evaluate them against more recent and accurate real-world data to verify if the patterns that we identified are still relevant to the threats that are prevalent in modern times.

Secondly, future work could be done on addressing the large amounts of false positives. High Recall is great for ensuring the financial protection of the bank, but the bank also needs to consider the satisfaction of their customers. If the security guard stopped everyone from coming in the building, they wouldn't have any bad actors in the building, but they wouldn't have any customers either.

Future efforts should begin exploring more advanced techniques in deep learning. Methods such as Neural Networks or Autoencoders for anomaly detections would better capture the more complex and non-linear patterns of fraud seen in this dataset and ultimately should reduce false flags.

Additionally, the implementation of more dynamic classification thresholds would give financial institutions flexibility to adjust their systems as necessary. Adjusting the balance of Precision-to-Recall ratio would enable businesses to modify their system based on real-time needs of the business or seasonal risk levels. For instance, it's known that around the 4th quarter of the year, consumers spend larger amounts on gifts and travel expenses. In that case, it makes sense to raise the thresholds as to not improperly flag well meaning folks who just happen to be spending more than normal due to the time of year.

Lastly, being able to integrate a real-time data pipeline, maybe using a live API, would make it possible to test these Models in a true production environment, adjusting from a static environment where we're just testing proof-of-concept frameworks to a truly living, adapting, and continuously learning fraud detection system.

6.2 Conclusion

It is evident when considering all the various types of Models that were implemented in this study, that the **Ensemble Methods** (especially Stacking multiple Models) were the optimal solutions for detecting fraud in a dataset that had such a large class imbalance. The Stacking Model from all eight base Models (Model 3) performed the best, achieving a **Recall** of **81.78%** and an **F1-Score** of **0.0897**. This evidence made it clear that combining several different types of learning algorithms, using meta-learners, incredibly outperforms an individual algorithm on its own. However, of those "standalone" Models, **Gradient Boosting** (Model 1 and 3) was the best-performing, achieving a **Recall** of **81.14%** and an **F1-Score** of **0.0903** with an **ROC-AUC of 0.8955**. Close behind Gradient Boosting was **Logistic Regression** (Model 1), which provided the most reliable handling of **Precision** and **Recall** on the full, imbalanced dataset, achieving an **F1-Score** of **8%** while maintaining an **80% Recall.**

The analysis we conducted highlighted a repeating observation; there was a consistent trade-off between Precision and Recall. From Model 2, additional insights were attained beyond pure predictions. The Decision Tree Model made it clear that the detection of fraud was greatly influenced by features such as transaction velocity, session, and financial features like credit risk score and credit limit. This Model was capable of achieving a high Recall of approximately 74%. The implication here is that this Model was able to capture a large portion of fraud cases, but, unfortunately, the Precision remained low (~3%). That is why the F1-Score dropped to just ~0.06. Comparing Model 2 to Model 3 shows exactly what the tuning fixed. Model 3 pushed the Recall up to 81.78%. It also had much better Precision. Because of this, the final F1-Score climbed to 0.14. This contrast shows the two different Modeling trade-offs. Model 3's approach aimed for a more balanced classifier, whilst Model 2 focused more heavily on capturing as much fraud as possible, even if false positive rates were high.

On the flip side,the unsupervised Models like K-Means clustering and Apriori association rule mining gave insights such as fraud not being randomly distributed. Despite the differences between the supervised and unsupervised approaches, both reinforced that fraud is driven by a mix of behavioral and identity-related factors, clustering around specific features such as housing status and device type. The takeaway from these observations is that a good fraud detection system must be able to look at both the classification prediction and the behavioral aspect of the transaction. If only predictive classification is used, the Model may very well miss new types of fraud that have not been labeled. If only behavioral segmentation is considered, the Model may miss an obvious one-time attempt since its focus is on long-term behaviors. Instead, a combination of both methods is recommended to maximize the efficiency of fraud detection.

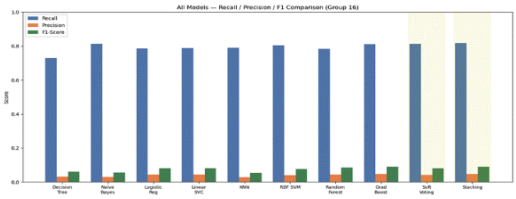

Figure 5: Model Comparison of Performance (Imbalanced Test Set)

References

[1] Rakesh Agrawal and Ramakrishnan Srikant. 1994. Fast Algorithms for Mining Association Rules. In *Proceedings of the 20th International Conference on Very Large Data Bases*. 487–499.

[2] Amazon Science. 2022. Fraud Dataset Benchmark (FDB). GitHub Repository. https://github.com/amazon-science/fraud-dataset-benchmark Retrieved April 2025.

[3] Siddhartha Bhattacharyya, Sanjeev Jha, Kishore Tharakunnel, and J. Christopher Westland. 2011. Data Mining for Credit Card Fraud: A Comparative Study. *Decision Support Systems* 50, 3 (2011), 602–613.

[4] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[5] Raghavendra Chalapathy and Sanjay Chawla. 2019. Deep Learning for Anomaly Detection: A Survey. *arXiv preprint arXiv:1901.03407* (2019).

[6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357. doi:10.1613/jair.953

[7] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794. doi:10.1145/2939672.2939785

[8] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. 2015. Calibrating Probability with Undersampling for Unbalanced Classification. In *Proceedings of the IEEE Symposium Series on Computational Intelligence*.

[9] Sérgio Jesus, José Pombal, Duarte Alves, André Cruz, Pedro Saleiro, Rita P. Ribeiro, João Gama, and Pedro Bizarro. 2022. Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation. In *Advances in Neural Information Processing Systems*, Vol. 35. https://proceedings.neurips.cc/paper_files/paper/2022/hash/d9696563856bd350e4e7ac5e5812f23c-Abstract-Datasets_and_Benchmarks.html

[10] Machine Learning Group – ULB. 2018. Credit Card Fraud Detection. Kaggle Dataset. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud Retrieved April 2025.

[11] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html